

Algoritmus

1. Co je to algoritmus?

Algoritmus je konečná posloupnost kroků (instrukcí), které vedou k vyřešení určitého úkolu nebo problému. Algoritmus popisuje postup přesně tak, aby jej mohl provést člověk i počítač.

2. Základní vlastnosti algoritmu

Konečnost (finitnost): Algoritmus musí skončit po konečném počtu kroků, nesmí běžet nekonečně dlouho.

Resultativnost (výslednost): Musí vracet výsledek nebo indikaci, že řešení neexistuje.

Jednoznačnost: Každý krok musí být přesně specifikován, při stejných vstupech musí vést ke stejnému výstupu.

Vstupnost: Musí být jasně definováno, jaké vstupy algoritmus očekává; neměly by se používat skryté/externí proměnné.

Obecnost: Algoritmus by měl řešit třídu problémů, ne pouze jednu konkrétní instanci.

Správnost: Algoritmus musí prokazatelně dávat požadovaný výsledek (důkaz nebo argumentace).

Efektivnost: Algoritmus by měl být časově a paměťově efektivní, ideálně s co nejmenší složitostí.

3. Typy algoritmů

Výpočetní: Provádějí numerické nebo symbolické operace (např. výpočet faktoriálu).

Třídící (sorting): Uspořádají data podle zvoleného kritéria (např. quicksort, mergesort, bubble sort).

Vyhledávací (searching): Hledají prvek v kolekci (lineární vyhledávání, binární vyhledávání).

Řadící (ordering): Obdobně jako třídící, někdy se termíny prolínají.

Iterační: Používají cykly (for, while) k opakování kroků.

Rekurzivní: Funkce volá samu sebe (např. rekurzivní faktoriál, quicksort).

Rozhodovací: Volí mezi různými postupy podle podmínek (if/else, switch).

4. Příklad — pseudokód a vysvětlení

4.1. Iterativní:

Pseudokód:

```
FUNCTION FactorialIter(n)
  if n < 0 then ERROR 'n musí být nezáporné'
  result ← 1
  for i from 2 to n do
    result ← result * i
  return result
```

4.2. Rekurzivní faktoriál:

```
FUNCTION FactorialRec(n)
  if n == 0 then return 1
  return n * FactorialRec(n-1)
```

5. Příklady třídících a vyhledávacích algoritmů

Krátce k nejznámějším:

Algoritmus	Typická časová složitost (průměr)	Poznámka
Bubble Sort	$O(n^2)$	Jednoduchý, ale neefektivní
Insertion Sort	$O(n^2)$	Dobře funguje pro malé nebo částečně setříděné množiny
Merge Sort	$O(n \log n)$	Stabilní, rekurzivní, využívá extra paměť
Quick Sort	$O(n \log n)$	Efektivní
Heap Sort	$O(n \log n)$	Nestabilní, dobré paměťové vlastnosti
Linear Search	$O(n)$	Prohledávání bez předpokladu o seřazení
Binary Search	$O(\log n)$	Vyžaduje setříděné pole